

SUMMER COURSE

Engineering Toolbox

Week 1 · 信任、一致性與可重現性

獵奇緯



目錄

01 課程地圖

03 Git：管理修改歷史，而不是管理檔案

05 curl：確認一個網路服務到底通到哪一層

07 收束

02 SSH：跟一台不在你面前的機器建立信任

04 GitHub：把本機的 Git 變成團隊的介面

06 Docker：把「環境」本身變成可以版本控制的東西

SECTION 01

課程地圖



01

四套工具，其實是一個問題

當開發不再是一個人在一台機器上寫程式，
要靠什麼機制維持信任、一致性和可重現性？



SSH

信任遠端機器



Git

信任修改歷史



HTTP / curl

定位服務故障



Docker

重現執行環境

✓ Tip

不照 lab 劇本走，每節結尾提對應 lab，操作留到 lab 時間。

I 完成這週，你應該能夠

建立信任

- 公開金鑰驗證、host key 與 identity key
- `ProxyJump` / bastion 架構
- Git 物件模型、branch、merge / rebase / conflict
- GitHub 如何把 repo 變成協作介面

診斷與交付

- 用 `curl`、DNS、TLS 分層定位服務故障
- container 與 VM 的本質差異
- 寫出安全、可重現的 Dockerfile 與 `compose.yaml`
- 審查 AI 生成部署設定是否安全、可重現、可信

SECTION 02

SSH：跟一台不在你面前的機器建立信任



為什麼會有 SSH

1995 之前

telnet / rlogin / rsh

帳號與內容都是明文

同網段側錄工具就能看光密碼

Tatu Ylönen, 1995

帳號被竊聽入侵後

寫出第一版 SSH

兩個承諾：加密整條連線 + 確認對方是誰



× 危險

只做到加密不夠：你可能在跟中間人「安全地」聊天，完全感覺不到差異。

I 私鑰簽名，公鑰驗證

Private key

留在自己的機器
用來簽名 challenge
絕不透過網路傳送

Public key

交給伺服器
用來驗證簽名
沒有機密性可言、無法反推私鑰

伺服器 challenge → 私鑰 sign → 公鑰 verify

✓ Tip

密碼每次登入都要「交出去」，金鑰簽名時私鑰從頭到尾沒離開過你的機器。

I 產生一組金鑰

terminal

```
ssh-keygen -t ed25519 -f ~/.ssh/lab
```

-t ed25519

比 rsa 短、快
抗側通道攻擊設計更嚴謹

passphrase

私鑰檔案本身的密碼
第二層防線

兩個檔案

lab 私鑰，權限必須

600

lab.pub 公鑰，可放心
公開

雙向信任：Host key 先上場

Host key

server → you



Identity key

you → server

terminal

```
$ ssh -p 2222 field@127.0.0.1
The authenticity of host '[127.0.0.1]:2222' can't be established.
ED25519 key fingerprint is SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

⚠ 注意

這一步在問：「這個 fingerprint 摸起來陌不陌生？」

TOFU：known_hosts 記住了什麼

👁️ 第一次

透過另一個 channel

(wiki、口頭、公司通訊軟體)

比對 fingerprint，不要無腦按 yes

📖 之後每次

記進 `known_hosts`

比對，fingerprint 變了

SSH 會嚇人地拒絕連線

✖ 危險

fingerprint 改變不是 bug，先查原因，不要刪掉那行 known_hosts 草草了事。

Identity key：你證明「我是我」

terminal

```
ssh-copy-id -i ~/.ssh/lab.pub -p 2222 field@127.0.0.1
```

做的事

用密碼登入一次

公鑰追加進遠端 `authorized_keys`

權限地雷

`authorized_keys` / `.ssh` 權限太寬

SSH 一樣直接拒絕使用

⚠ 注意

一次連線，兩次驗證：host key 證明「我是你要連的機器」+ identity key 證明「我是被允許登入的人」，任一邊被冒充，信任鏈就破了。

ssh-agent：解鎖一次，委託簽名

```
terminal  
  
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/lab
```

ssh-agent

常駐小程式，解鎖一次私鑰進記憶體
同 session 後續連線直接向它借用

agent forwarding `-A`

傳出去的是「簽名能力」的委託
跳板被入侵，攻擊者可冒充你連下一台

× 危險

不是私鑰外洩，但效果幾乎一樣，沒有明確需求就不要開 forwarding。

! /.ssh/config 與 ProxyJump：別名與唯一入口

```
~/.ssh/config

Host field-gateway
    HostName 127.0.0.1
    Port 2222
    User field
    IdentityFile ~/.ssh/lab
Host robot-01
    HostName robot-01
    User operator
    ProxyJump field-gateway
```



終端機上只打一行 `ssh robot-01`，底層走的是兩段獨立、各自驗證身份的 SSH 連線。

重用同一條隧道：檔案傳輸與埠轉發

terminal

```
scp -P 2222 report.txt field@127.0.0.1:/home/field/  
rsync -avz -e "ssh -p 2222" ./deployment/ field@127.0.0.1:/opt/robot/config/  
ssh -L 8080:localhost:80 field-gateway
```

scp / rsync

已有被信任的管道，就不再開新的門

`rsync` 增量比對，部署腳本首選

port forwarding `-L`

服務只監聽 `127.0.0.1`

「誰能碰到它」的責任交給 SSH 這層

SECTION 03

Git：管理修改歷史，而不是管理檔案



03

Git 管理的不是檔案，是「有意圖的歷史」

直覺以為

更潮的 Ctrl+Z
自動備份資料夾

實際上是

有意圖、有作者、有時間
彼此有因果關係的修改歷史

⚠ 注意

2005，BitKeeper 授權糾紛，Linus 十天寫出雛形——痛點來自管理上萬貢獻者同時修改同一批檔案。

分散式

分支零成本

密碼學完整性

大規模效能

內容定址的物件資料庫：四種物件

blob

檔案內容
不含檔名

tree

目錄
檔名 → blob /
tree

commit

完整快照
tree + parent +
metadata

tag

commit 的
人類名稱

Info

理解物件模型後，之後所有指令的行為都能直接推導出來，不用死背。

Tip

每個物件用內容雜湊當 ID：改一個 bit 就是全新物件。commit 是「一串快照」而非「一串差異」，未變的 blob 被原樣重用。

分支是會移動的指標，歷史是一個 DAG

`.git/refs/heads/main` = 一個 commit SHA

HEAD → main → commit

timeline

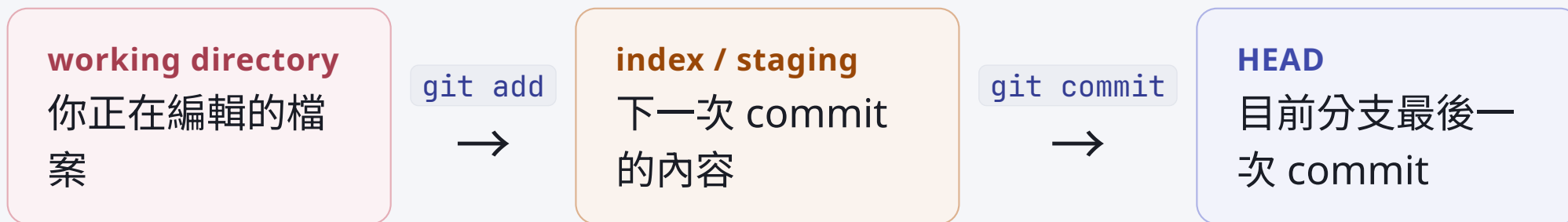
DAG

merge = two parents

Info

開分支瞬間完成，只是多寫一個指標；`git log --graph` 的分岔線畫的就是這個圖本身。

三棵樹：working directory / index / HEAD



❶ Info

`git status` 報告的永遠是這三者兩兩之間的差異。

merge 的兩種走法

fast-forward

main 分支後完全沒有新 commit
指標直接往前挪，連 merge commit
都不用生

三方合併

兩邊都往前走了（git-merge-lab 場景）
找共同祖先，比對兩邊各自的差異

改不同地方

Git 有把握自動套用兩份修改
生出 two-parent merge commit

改同一處

Git 無法決定留哪份意圖
→ 衝突

衝突：Git 如實呈現，人負責意圖

```
velocity_controller.cpp  
  
<<<<<< HEAD  
main 分支目前內容  
=====  
incoming branch 內容  
>>>>>> feature/safety-limit
```

⚠ 注意

衝突不是「合併失敗」，是 Git 老實承認不知道兩段意圖怎麼揉合，如實呈現給人看。

| rebase：換一種改寫歷史觀感的方式

merge

保留分岔又交會的痕跡
真實發生的樣子

rebase

commit 重新「演」一遍
產生全新 SHA，歷史看起來像從沒分岔過

× 危險

未推送的本機分支：隨意 rebase。已推上去、別人可能已拉下去的歷史：不要 rebase。

SECTION 04

GitHub：把本機的 Git 變成團隊的介面



04

remote：一份可推拉的另一份歷史

Git 是分散式的，*GitHub* 是讓團隊形成共識的場所，不是「正本」。

terminal

```
git remote add origin git@github.com:team/robot-controller.git
git fetch origin
git push origin main
```

fetch

抄一份歷史下來
不動本機工作分支

pull

fetch + merge/rebase
悄悄做了兩件事

push

送出本機領先 commit
走的正是 SSH

■ Pull Request：把「要不要合併」變成可討論的決策

protected branch

禁止直接 push
main

required review

至少一位他人核
准

CI

建置與測試必須
過

code owner

特定路徑由負責
人核准

I AI 協作者進場之後，這層介面的意義變了什麼

舊假設

審查者信任提交者對程式碼有基本理解



AI 打破的假設

diff 可能是幾秒前生出來的，提交者本人都還沒完整讀過

× 危險

AI 生成的 diff，審查標準不能比人寫的更鬆，反而該更謹慎。

SECTION 05

curl：確認一個網路服務到底通到哪一層



這一節不是為了背指令

老實講

ROS 2 講 DDS
不是 HTTP
以後不太會常打

真正要教的

curl 背後的協定
分層除錯思路

瀏覽器

幫你藏掉
太多細節

curl -v

攤開 IP、port、
TLS、
header、status

❗ Info

之後 visual-relay 的健康檢查完全不靠 curl，直接用 nc 手打 HTTP request，看懂前提是先把這裡搞懂。

I 拆開一個 URL

```
https://api.example.com:8443/v1/robots?id=42
```

scheme

https

host

需 DNS

port

8443

path

/v1/robots

query

id=42

✓ Tip

把 URL 拆開，除錯才有順序。

CURL：確認一個網路服務到底通到哪一層

| DNS：把名字變 IP

```
terminal  
  
getent hosts api.github.com  
dig api.github.com
```

getent hosts

系統真正的解析路徑

`/etc/hosts`、DNS、mDNS

dig

比較像直接問 DNS

TCP port：三個常被混淆的地址

terminal

```
curl -v http://localhost:8080/health  
curl -v http://127.0.0.1:8080/health  
curl -v http://0.0.0.0:8080/health
```

127.0.0.1

loopback，只指自己

localhost

通常解析到 loopback

0.0.0.0

bind 時的「所有網卡」
當 client 目標語意混亂

⚠ 注意

Docker 裡 container 的 localhost 是 container 自己，host 的 localhost 是 host 自己，常見坑是站錯 namespace。

| HTTP method 與 status code

method

GET 拿 · POST 送

PUT/PATCH 更新 · DELETE 刪

status code

2xx 成功 · 3xx redirect

4xx client 問題 · 5xx server 問題

4xx

404 Not Found 、 401 Unauthorized

5xx

500 Internal Server Error 、 503 Service Unavailable

I 常用 curl 指令

```
terminal  
  
curl -I https://example.com  
curl -v http://localhost:8080/health  
curl -sS http://localhost:8080/status  
curl -X POST http://localhost:8080/api \  
  -H 'Content-Type: application/json' -d '{"speed": 0.2}'
```

-I header

-v 細節

-sS 適合 CI

-H / -d

沒裝 curl，也能手打 HTTP

```
healthcheck-service.sh  
  
nc -w 1 127.0.0.1 "$port" <<EOF | grep -q 'ready=true'  
GET /ready HTTP/1.1  
Host: 127.0.0.1  
Connection: close  
  
EOF
```

Info

nc 只把文字原封不動送進 TCP，懂 HTTP 格式的是寫這段文字的人，還有另一端的 server。

TLS 與身份：憑證、token

terminal

```
openssl s_client -connect api.github.com:443 -servername api.github.com  
curl -H "Authorization: Bearer $TOKEN" https://api.example.com/v1/me
```

⚠ 注意

`certificate verify failed` = 「我無法用本機信任的 CA 鏈驗證這張憑證」。

`-servername` 是 SNI，少了它伺服器可能給你另一張憑證。

✗ 危險

token 是權限：進 secret store、限制 scope、避免出現在 shell history、log、commit 裡。

CURL：確認一個網路服務到底通到哪一層

I jq 切開回應，照層次除錯

terminal

```
curl -sS https://api.example.com/v1/robots | jq '.items[] | .name'
```



✓ Tip

Compose 裡 service name 靠 DNS 找彼此，healthcheck 常用 HTTP endpoint 判斷 readiness。

| wget 與同一排工具

curl

跟一個 endpoint 對話
header、method、token、JSON、
debug

wget

把檔案抓下來
release tarball、遞迴抓站、斷點續傳

terminal

```
wget -c https://example.com/large-dataset.zip  
wget -r -np https://example.com/docs/
```

SECTION 06

Docker：把「環境」本身變成可以版本控制的東西



「我的機器上可以」：兩條解法的路

works on my machine

OS 版本 · 函式庫版本 · 環境變數 · 系統套件 · 殘留設定檔，原因從來不是單一個。

第一條路：VM

模擬整台假硬體，跑獨立 kernel
隔離徹底，代價徹底：開機慢、映象幾 GB

第二條路：container

隔離的「世界」共用 host kernel
Docker 只是工具鏈之一：Podman 也能
跑同一個 image (daemonless / rootless)

macOS / Windows 怎麼跑 Linux container

macOS

Docker Desktop / Colima / Lima
幫你管理底下那台 Linux VM

Windows

Docker Desktop 長期走 WSL2
backend
借 WSL2 的 Linux kernel

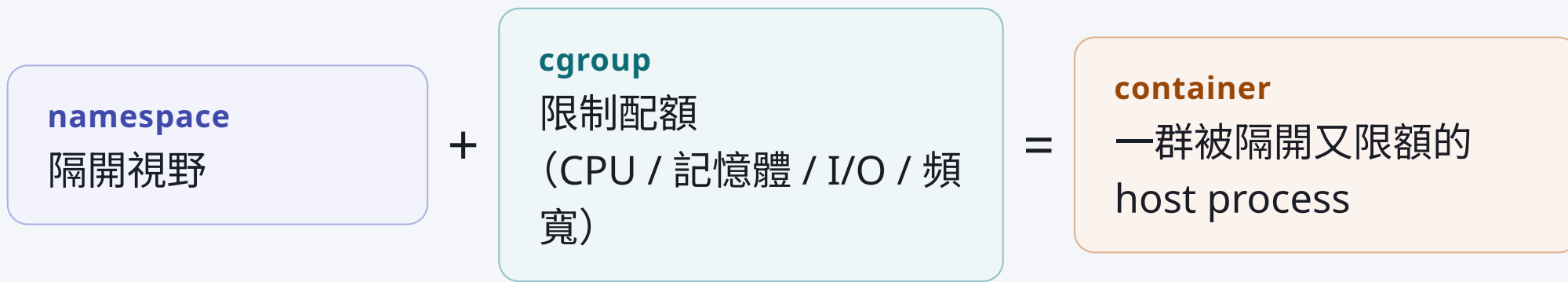
Info

近年更系統化：Apple `container` / Containerization、Microsoft WSL Containers（`wslc.exe`）。核心共通概念才是重點：image、registry、namespace、cgroup、mount、network、權限。

I namespace：這個 process 看得到什麼

PID	container 裡第一個 process 是 PID 1，即使 host 上其實是 PID 88213
mount	獨立的檔案系統樹，跟 host 或其他 container 互不相干
network	自己的網卡、IP、routing table、埠空間
UTS	自己的 hostname
user	container 的 root 可對應 host 上完全沒特權的一般使用者

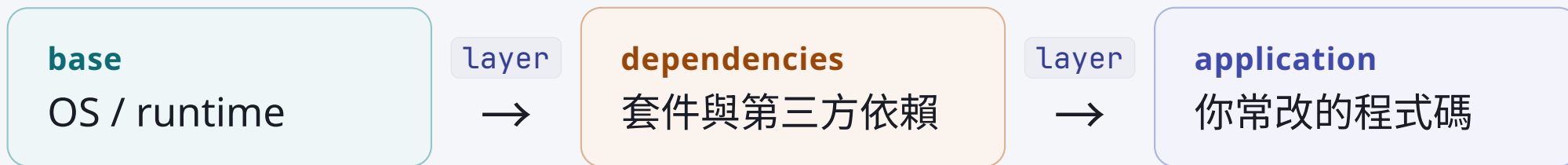
I cgroup：這個 process 能用多少



⚠ 注意

container 之間共用同一個 host kernel，kernel 本身的漏洞能跨越這層隔離，跟 VM 是不同等級的隔離強度。

Image layer：內容定址的既視感



✓ Tip

跟 Git 物件模型似曾相識，每層用自己內容的雜湊當 ID，輸入沒變就重用上次 build 的 layer。

讓 build 吃到快取

改動頻率低的排前面，改動頻率高的排後面

前面

系統套件、編譯工具、第三方依賴

後面

`src/`、`include/`、測試檔

layer cache

快取「這層執行完的檔案系統結果」

cache mount

快取「工具自己的工作目錄」

apt / ccache / pip，靠

```
# syntax=docker/dockerfile:1.7
```

multi-stage build：建置期與執行期分開

```
Dockerfile

FROM build-image AS builder
RUN make release

FROM runtime-image
COPY --from=builder /app/bin/service /usr/local/bin/service
USER app
CMD ["service"]
```

× 危險

新手常把編譯器、原始碼、測試框架一起打進 production image——又大又肥，還多了攻擊面。visual-relay 起始版本故意這麼糟；「能 build」不等於「設計完成」。

runtime 身份與最小權限

⚠ 預設

container process 是 root

✓ better

專用非 root 帳號：不需登入 shell、不需密碼、不需管理系統

檔案

read-only filesystem
明確暫存例外

權限

no-new-privileges
禁止中途提權

資源

CPU · memory · PIDs
上限

📄 Info

不是為了讓 YAML 好看，是最小權限原則：預設關閉一切，只開明確需要的門。

GUI container：一條很窄的門

host session

一組 process、socket、環境變數、權限

`DISPLAY` / `WAYLAND_DISPLAY`

X11 除錯方向

較老、較寬鬆：`DISPLAY`、
Xauthority、socket mount、權限

container 預設

看不到 host socket、未必有權限連

→ `cannot open display`

Wayland 除錯方向

較新、權限收斂：`runtime socket`、
`XDG_RUNTIME_DIR`、portal、支援度

| network：service name 就是 DNS

camera-net

handoff-net

control-net

⚠ 注意

三張隔離的網，不是一張大平面網路。一個 container 只加入它真正需要通話的那幾張，能力邊界應跟職責邊界重合，跟 SSH 的 ProxyJump 是同一種設計哲學。

healthcheck：活著不代表健康

process 活著

`docker ps` 照樣顯示在跑
就算卡死在無窮迴圈裡

≠

服務真的健康

healthcheck 定期執行檢查指令
退出碼 → healthy / unhealthy

`interval` · `timeout` · `retries` · `start_period`

compose：把系統描述成一份檔案

我要這幾個 *service*，接這些 *network*，用這些 *volume*，帶這些限制。

services

要跑哪些東西

networks

通訊平面
讓不同職責流量分開

volumes

資料生命週期

⚠ 注意

少一個參數，系統可能看起來「有跑起來」，行為已偏離設計——部署從手動流程變成可重建的系統描述，跟 systemd unit、Kubernetes 相通。

depends_on 是順序，不是 readiness

depends_on

process 起來了

⇒ schema 初始化完成 / 能正常回應 /
拿到 display

healthcheck

更接近「服務真的可用」的訊號

Info

Compose 適合單機多服務：本機開發、作業、demo、CI job。跨機器排程、rolling update 交給 Kubernetes 這類 orchestrator。

SECTION 07

收束



07

回到開場的問題

當開發不再是一個人在一台機器上寫程式，
要靠什麼機制維持信任、一致性和可重現性？

SSH 的答案

信任要驗證
不要假設
雙向金鑰、
TOFU

Git 的答案

一致性來自
不可篡改的歷史
內容定址、DAG

curl 的答案

故障要分層定位
DNS→TCP→
TLS→HTTP

Docker 的答案

環境要能
被描述、被重建
namespace、
compose

帶進 lab，也帶進 AI 時代

🔧 lab 的驗收不只是跑通

它信任誰？邊界在哪？

失敗怎麼被發現？

換一台機器能重現嗎？

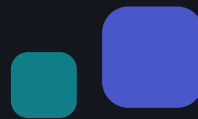
🤖 AI 協作的把關

AI 能生 diff、Dockerfile、compose

判斷「安全嗎、可重現嗎、可信嗎」

的人是你





Q & A

獵奇緯 · Summer Course